REDES (RCCD) — MACHETE 2 · Guías 5–8 + teóricas 12–15

1 · CONGESTIÓN (G5)

Valores guía: SMSS=2KB · IW=2·SMSS=4KB · LW=1 SMSS (post time-out) · RW=min(IW,CWND) (post idle ≥1 RTO) · ISSTHRESH=64KB.

Se envía min(CWND, RWND) por RTT (mínimo, no máximo; siguen independientes). **RWND**=AdvWin=espacio libre del buffer del receptor (**flujo**, receptor, punto a punto). **CWND**=estimación del emisor de lo que aguanta la **red** (**congestión**, global). EffWin=MaxWin−(LBS−LBA); **FlightSize=LBS−LBA** ⇒ **LBA=LBS−Flight**.

Situación	SSTHRESH	CWND
SS (CWND< SSTH)	=	+min(N,SMSS) por ACK ⇒ **×2/RTT**; topear en SSTH
CA (CWND≥ SSTH)	=	+SMSS²/CWND por ACK ⇒ **+1 SMSS/RTT**
Time-out	max(**Flight**/2, 2SMSS)	1 SMSS → SS
3 dup ACK (4 iguales)	max(Flight/2, 2SMSS)	SSTH+3SMSS; +1 SMSS por dup extra
ACK nuevo en FR/FR	=	=SSTH → CA

SSTHRESH sale de FlightSize/2 (lo enviado=min(CWND,RWND)), **no de CWND/2**. **Ráfaga perdida entera ⇒ no vuelve ningún ACK ⇒ time-out** (nunca FR/FR). Por qué +1/RTT en CA: (CWND/SMSS ACKs)·(SMSS²/CWND)=SMSS; en SS: W ACKs·1=+W ⇒ ×2. N=bytes *nuevos* que reconoce el ACK (dup=0); min(N,SMSS) evita ACK division. SSTH sólo cambia ante pérdida (puede subir). Corte a la mitad: vaciar la cola + equidad AIMD (Chiu-Jain) + conservador.

Método trazas: tabla RTT | CWND | envía=min(CWND,RWND,falta) | ¿≥umbral? | SSTH | acumulado. Ráfaga perdida + RTO (=2RTT, timer desde el envío) = **2 RTTs**. Estacionario = SSTH deja de cambiar ⇒ ciclo: total=(datos/bytes por ciclo)×RTTs por ciclo. Supuestos: duración TO, borde CWND=SSTH, 1KB=1024, 1 ACK/segmento.

Leer tablas (E12): ×2=SS · +1 SMSS=CA · cae a 1 SMSS=TO · cae a ~mitad(+3)=FR/FR · Flight<CWND ⇒ limita RWND o la app · LBS baja ⇒ reTx desde LBA · LBA quieto + CWND sube ⇒ dup ACKs (inflado FR/FR).

E9 (RWND 64, pierde ráfaga ≥X): X=32 ⇒ SSTH=16, ciclo 2+4+8+16+18..30=198KB/13RTT≈15,2KB/RTT; 1GB≈**68 800 RTT**. X=16 ⇒ SSTH=8, 50KB/8RTT=6,25 ⇒ **167 800 RTT** (×2,4, no ×2: el costo fijo pesa más). Sin cong.: 64KB/RTT ⇒ **16 388 RTT**.
E10 RWND 16, pierde ≥16, 40KB: 4,8,*X* 16,RTO,2,4,8,10,4 ⇒ 9 RTT=1,8s; CWND final 12 (≈12,67).
E11 pierde ≥20. a) RWND 28: 1er TO pierde min(32,28)=28 ⇒ SSTH=**14** (no 16); 2º pierde 20 ⇒ **10** (estacionario, 13 RTT≈650ms); ciclo 84KB/10RTT≈8,4. b) RWND 18 < 20 ⇒ **nunca pierde** ⇒ **18KB/RTT: más rápido**.
E8 (1 pérdida/s, 10 RTT/s): sin FR/FR SSTH→14; con FR/FR → 20 (sierra 20–38, AIMD puro).

RTT/RTO: RTT=segmento→ACK ≠ T_{tx}=SMSS/BW. Est=α·Est+(1−α)·S; RTO=2·Est. α alto (.8–.9) estable; α bajo ⇒ RTO se desploma ⇒ **TO espurio**. Chequear S>RTO vigente *antes* de actualizar. **Karn**: no samplear reTx; RTO×2 al reTx. **Jacobson**: Diff=S−Est; Est+=δDiff; Dev+=δ(|Diff|−Dev); RTO=Est+4Dev (δ=1/8).

Teórica

Congestión=sobrecarga **sostenida**, demanda ≥ capacidad ⇒ QoS degradada (pérdidas + delay creciente). **Knee**: throughput deja de crecer, delay se dispara. **Cliff**: colapso, goodput→0. **Potencia**=throughput/delay (máx≈knee). Evitar=operar en knee; controlar=no caer del cliff.

FIFO (scheduling: quién sale) + **Drop Tail** (buffer: se tira el que llega) ⇒ **sincronización global**. **RED**: Avg=(1−w)Avg+w·Sample; <MinTh encola, entre umbrales descarta con P (hasta MaxP≈0,02), ≥MaxTh descarta; MaxTh=2MinTh ⇒ avisa temprano a flujos al azar.

Jain f=(Σx)²/(n·Σx²) ∈[1/n,1]. **M/M/1** (Poisson, servicio exp, 1 server): ρ=λ/μ<1; E[N]=ρ/(1−ρ), **E[T]=1/(μ−λ)**→∞ con ρ→1. **Mathis** BW=(MSS/RTT)·√(3/2)/√p (penaliza RTT largo/pérdidas; WiFi pierde por bits y TCP lo malinterpreta).
No alcanzan: buffers ∞ (delay→TO→duplicados), ruteo (sólo reparte; si mira carga oscila), más BW (corre el problema), prealocar (datagramas: no hay circuito). **Taxonomía**: lazo abierto (circuitos) / cerrado: implícito (TCP, pérdidas) vs explícito (ECN, DECbit, ATM ABR). SS es "slow" vs TCP pre-1988 (mandaba toda la AdvWin). CWND no representativa: errores de bit, idle, app-limited.

2 · APLICACIÓN (G6)

Orden: nombre → **DNS (UDP 53)** → IP → **TCP** (1 RTT handshake) → protocolo app. Integradores: repetir por **cada host nuevo**. Puertos: HTTP 80 · HTTPS 443 · SMTP 25 (587 submission) · POP3 110 · IMAP 143 · DNS 53/UDP.

1 server/1 IP/1 puerto ⇒ N clientes: TCP distingue por **4-upla**, cliente usa puerto efímero. Escalar = capacidad: **DNS round robin** (varios A; grueso, TTL/caché, no detecta caídas) vs **balanceador** (fino + health checks, punto único).
Protocolo app: tipos de msg, sintaxis, semántica, reglas; abiertos (RFC) o propietarios. Proceso=IP+puerto (socket). Cliente habla primero; server espera en puerto conocido.

App	Pérdidas	BW	Delay	Transp.
archivos/mail/web	no	elástico	no	TCP
audio/video RT	tolera	mín. kbps–Mbps	~100ms	UDP
video almacenado	tolera	mínimo	pocos s	TCP (DASH)
juegos	tolera	pocos kbps	~100ms	UDP/TCP

TCP da conexión, confiabilidad, flujo, congestión; **no da temporización ni BW mínimo**. UDP no da nada pero no agrega handshake ni reTx que llegan tarde.

DNS

BD distribuida jerárquica (raíz→TLD→dominio), escala por **delegación de zonas + caché**; administra ICANN. App llama gethostbyname → UDP al server local. **Stub resolver** (PC, no resuelve) →**recursiva**→ **resolver** (ISP, 8.8.8.8) →**iterativas**→ raíz → TLD → … → autoritativo. El resolver arranca del **ancestro más cercano en su caché** (si no, raíz); .ar no sirve para .com. **Cada resolver tiene su caché**.
RR `nombre TTL IN tipo valor`: A/AAAA · **CNAME** (alias, seguir cadena; con CNAME no hay otros registros; apex no puede ser CNAME) · **MX** (prioridad, menor=mejor) · NS · SOA · PTR.
NS: (1) en la zona **padre** = **delegación** (referral); (2) en la zona = lista de autoritativos (aa). Varios por redundancia. DNS en otro gestor ⇒ cargar sus NS en el registrar. "Propagación" = TTL de NS viejos. **Glue**: A del NS en la zona padre cuando el NS está **dentro** de la zona delegada (si no, circular); va en ADDITIONAL.
Primario = 1er nombre del SOA; resto de NS = secundarios (copian si sube el **serial**). SOA: serial, refresh, retry, expire, minimum (=TTL caché negativa).
Autoritativa (aa) vs no autoritativa (caché). Secciones ANSWER / AUTHORITY (NS) / ADDITIONAL (A extra). Cambio de IP: los cachés dan la vieja hasta que vence el TTL.
Zone file: nombre sin punto final = relativo (`ads` en jmail.com=ads.jmail.com; `cdn.akamai.com` sin punto=cdn.akamai.com.jmail.com). Server no autoritativo para un dominio no se consulta por él ⇒ esos registros no se usan.
Secuencias: tabla # | de→a | recursiva/iterativa | respuesta (referral NS+glue / aa A). 2ª resolución en .com arranca del NS de com (salta raíz). Mail masivo: **1 consulta MX por dominio distinto**, no por destinatario (A del MX en ADDITIONAL). DNS centralizado/sin caché/con conexión ⇒ cuello + punto de falla + RTTs extra (afecta cada HTTP; SMTP menos, encola).

E4 (Thunderbird gmail → dc.uba.ar, abrir www.smn.gov.ar; cachés vacías): TCP mín. **3**: PC↔www.smn (80, HTTP) · PC↔smtp.gmail.com (587) · smtp.gmail↔mx.dc.uba.ar (25; no la abre la PC). UDP: sólo DNS: PC↔resolver-isp; resolver-isp↔raíz, ar, gov.ar, smn.gov.ar, com, gmail.com; smtp.gmail↔resolver-google↔raíz, ar, uba.ar, dc.uba.ar.
E10 spam a @uba.ar, @dc, @dm, @df (desde caché): 1 A del saliente + 4 MX = **5 consultas** (9 si los A no vienen en ADDITIONAL).
E11 www.jmail.com + logo cdn.akamai.com + iframe ads.spameitor.com: **3 TCP** (1 por host; con 1.0 también 3 porque hay 1 objeto por host).

Mail

Saliente = SMTP de **tu proveedor** (smtp.gmail.com:587): autentica, encola, reintenta. ≠ entrante/MX (del **destinatario**) ≠ POP3 (baja) / IMAP (queda en server).
El MX lo busca el saliente con *su* resolver; la PC sólo resuelve el A de su saliente.
UA ─SMTP→ saliente ─(DNS MX+A)─SMTP→ MX destino → buzón ─POP3/IMAP→ UA. Webmail: navegador ─HTTPS→ webmail (SMTP por dentro). MX existe porque el mail va a otra máquina que la web (dc.uba.ar: web UBA, mail Google).
SMTP: TCP 25, server→server directo, 3 fases (saludo, transferencia, cierre). `HELO · MAIL FROM · RCPT TO · DATA · (cuerpo, termina en ".") · QUIT`; resp 250 OK, 354 empezá cuerpo, 221 cierre. RFC 822: To/From/Subject + línea en blanco + cuerpo ASCII. IMAP: carpetas, sincroniza dispositivos.
MIME: SMTP ASCII 7 bits ⇒ binario en **base64**; headers MIME-Version, Content-Type (qué es), Content-Transfer-Encoding (cómo). HTTP reusa los *tipos* MIME pero el cuerpo va binario (sin CTE).

HTTP

Request: `GET /ruta HTTP/1.1` + **Host obligatorio** (virtual hosting) + línea en blanco. HEAD=headers sin cuerpo. `If-Modified-Since` → **304**. Respuesta: `HTTP/1.1 200 OK` + Content-Type (lo decide el server, no la extensión) + Content-Length + blanco + cuerpo. POST upload → **201 Created** + Location. 1.1 agrega PUT, DELETE. Códigos: 200 · 201 · 301 Moved (+Location) · 304 · 400 · 401 · 404 · 505 Version Not Supported.
Sin estado (con estado: si uno se cae hay que recomponer estados inconsistentes). Estado afuera: **cookies, 4 partes**: (1) Set-Cookie en respuesta, (2) Cookie en pedidos siguientes, (3) archivo en el host (navegador), (4) BD del sitio indexada por ID; costo privacidad/rastreo. Auth en cada pedido (Authorization:; sin ella 401 + WWW-Authenticate).
Se descarga solo: `<img src> <link rel=stylesheet> <script src> <iframe src>` (otra página ⇒ sus recursos) `<input type=image>`. **No**: `<a href>`, `<form action>`. **Una conexión TCP por host** distinto (con su DNS previo; DNS es UDP). Pixel de rastreo: `<img src=count.php?Email=..>`.

RTTs/página (objeto=1 RTT handshake + 1 RTT GET; embebidos tras parsear HTML): 1.0 no persistente: **2n** (paralelo: 4) · 1.1 sin pipelining: **2+(n−1)** · con pipelining: **2+1=3**. **E6** (HTML+css+2 png+input img, n=5): 10 / 6 / 3 RTT. Objeto = 2RTT + T_{tx}.

CDN: servers que cachean contenido dentro/cerca de ISPs; reparte por **redirección DNS**: el DNS de la CDN devuelve la IP más cercana según desde dónde pregunta el *resolver*. **DASH**: video por HTTP en segmentos, el cliente elige bitrate.

3 · SEGURIDAD (G7)

Propiedades: confidencialidad · integridad · autenticación · **no repudio** (algo que *sólo* A pudo generar ⇒ firma con privada; ⇒ autenticación, no al revés; **MAC no da no repudio**: B tiene la clave). +**originalidad** (no es copia, anti-replay) · **temporalidad** (no demorado) · **control de acceso** (quién hace qué) · **disponibilidad** (contra DoS). Por capa: app SSH, PGP · transp. TLS · red IPsec · enlace WEP/WPA/WPA2.

Mi privada = probar que soy yo (firmar). Pública del otro = que sólo él lea (cifrar). Simétrica (AES): rápida, problema = distribuir (N(N−1)/2 claves). Asimétrica (RSA): lenta, resuelve distribución ⇒ **híbrido**: asimétrica autentica + acuerda simétrica de sesión.

Firma+cifrado (E2): F=E_{K-A}(H(M)); C=E_{K+B}(M+F). B: descifra C con K−B → M,F; F con K+A → digest; **recalcula H(M) y compara** (el hash no se invierte).
MITM (E3): sin cert, el atacante cambia las públicas y descifra/recifra. **X.509**=(sujeto, **K+ del sujeto**, emisor, validez, serie, algoritmos) + **firma de la CA**. Verificar: firma con **K+CA** (preinstalada = raíz de confianza), fechas, nombre, no revocado (CRL/OCSP). Cadena raíz → intermedia → server.
Clave compartida (5 msgs): A→B "soy A"; B→A R_B; A→B $K_{AB}(R_B)$; A→B R_A; B→A $K_{AB}(R_A)$. Nonce largo (128b) ⇒ anti-replay. Versión de 3 msgs ⇒ vulnerable a **reflexión** (T abre 2ª sesión mandándole a B su propio R_B).

Parcial G7 (E6–E9): ¿a quién hay que autenticar?
E6 proxy autentica usuarios ⇒ TLS mutuo con CA de la empresa. Proxy: su par + $Cert_P$ + K+CA. Cada usuario: su par + $Cert_U$ (firmado por la CA) + K+CA. Proxy manda CertificateRequest; usuario Certificate + CertificateVerify; proxy chequea firma CA, vigencia/CRL, CertificateVerify (tiene la privada) y si está autorizado. No user/pass.
E7 hotel (autenticidad del proxy + confidencialidad) ⇒ TLS normal: proxy con par + $Cert_P$ de una CA cuya K+CA ya está en los dispositivos (CA pública; los huéspedes no instalan nada). Cliente valida el cert (un proxy falso no tiene la privada) y manda la PMS con K+P ⇒ el que espía el WiFi no lee. Clientes sin cert.
E8 no repudio interno ⇒ cada empleado par + cert de CA interna; firma cada mensaje $F=E_{K-E}(H(M+ts))$; se guarda M+F en un log; cualquiera verifica con el cert. Simétrica/MAC no sirve (el otro tiene la clave).
E9 API sin cifrar todo ni sesiones: cliente tiene su privada; server tiene Cert/K+ de cada cliente. URL: `?method=getPriceAuth&fruit=Pera&id=33&ts=..&nonce=..&sig=`E_{K-C}`(H(cadena canónica))`. Server: busca K+C por id, recalcula H y compara, chequea ts reciente + nonce no usado (anti-replay), guarda pedido+firma (no repudio). V0 nada → id → pass (en claro) → MAC (sin no repudio) → firma (replay) → firma con ts+nonce ✓.

Challenge-response asimétrico: A manda **nonce**; B lo firma con K−B; A verifica con K+B. **Replay**: reusar msg viejo válido ⇒ nonce/timestamp *dentro* de lo firmado.
Reglas anti-reflexión: inicia el que prueba primero, claves distintas por sentido, challenges de conjuntos distintos. **KDC**: A→KDC $A,K_A(B,K_S)$; KDC→B $K_B(A,K_S)$; resuelve N² claves, pero punto central de confianza/falla.

SSH (E5)	Cliente	Servidor
a) autenticar clientes	su privada	N públicas en `authorized_keys`
b) autenticar server	K+S en `known_hosts` (1ª vez fingerprint/**TOFU**, débil)	su privada
c) con CA	su par + $Cert_C$ + **K+CA**	su par + $Cert_S$ + **K+CA**

Con CA no se guardan públicas ajenas: el cert llega en el handshake y se valida con K+CA. Revocar ⇒ CRL.

TLS 1.2 (E4)	Mensajes
HELLO	ClientHello (versión, cipher suites, $\mathbf{R_C}$) · ServerHello (elige suite, $\mathbf{R_S}$)
Key exchange	Certificate ($Cert_S$) · [CertificateRequest] · ServerHelloDone · ClientKeyExchange (**PMS cifrado con K+S**) · ChangeCipherSpec · Finished
Data	todo simétrico + MAC

master=PRF(PMS,"master secret",R_C+R_S); **nonces ⇒ sesión distinta ⇒ anti-replay**. Finished=MAC de todo el handshake. **TLS mutuo**: CertificateRequest → cliente manda Certificate + **CertificateVerify** (firma del handshake). Autenticar empleados ⇒ TLS mutuo con CA de la empresa. Capas: TLS (app/TCP) · IPsec (red: **AH**=integridad+aut, no cifra; **ESP**=confidencialidad) · WPA (enlace). **Kerckhoffs**: algoritmo público, clave secreta. Factor de trabajo exponencial en el largo de clave. **OTP** inviolable (Shannon) pero impráctico. Sustitución (enmascara, preserva orden): César (rotación; probar 25 y ver cuál da texto legible), monoalfabética ($26!\approx4\cdot10^{26}$ claves, cae por frecuencias), polialfabética. Transposición (reordena): clave sin letras repetidas (MEGABUCK) numera columnas, se escribe por filas y lee por columnas. Cifrado≠código (Navajo). Clave 64b juguete, 128/256 comercial. DES 64b/16 rondas, 3DES=E·D·E, AES hoy. Hash: muchos a uno, tamaño fijo, avalancha; preimagen / 2ª preimagen / colisión; MD5(128), SHA-1(160) rotos, SHA-2 vigente. **K−(K+(m))=m=K+(K−(m))**: pública primero=confidencialidad, privada primero=firma (autenticidad+integridad+no repudio; **no da confidencialidad**).

RSA: n=pq, z=(p−1)(q−1), e coprimo con z, **ed mod z=1**; $c=m^e \bmod n$, $m=c^d \bmod n$. Ej p=3,q=11: n=33, z=20, e=3, d=7.

4 · BLOCKCHAIN

Problema: **doble gasto** sin autoridad central (firma no da orden/unicidad/acuerdo). Ledger replicado; bloque = **hash del anterior** (cambiar uno rompe los siguientes) + raíz **Merkle** (inclusión en O(log n)). Tx firmadas con privada del dueño (dirección = hash de la pública). **Sybil**: identidades gratis ⇒ votar con recurso escaso.
PoW: nonce con H(bloque)<target (caro hacer, barato verificar); gana la cadena con más trabajo; **51%** reescribe historia; confirmar ≈ esperar k bloques; dificultad cada 2016 bloques (~10 min); gossip + forks transitorios (huérfanos). Atacante con q<50% ⇒ chance de alcanzar cae exponencial con z bloques; "6 confirmaciones (~1h)" es convención. Bitcoin ~7 tx/s; bloques más rápidos ⇒ más forks. Gastar = referenciar UTXO + firmar (ECDSA). "La firma dice quién; el consenso, cuál primero."

	PoW	PoS	BFT (PBFT)
Recurso	cómputo	capital (slashing)	identidad conocida
Red	abierta	abierta	permisionada
Finalidad	probabilística	económica	inmediata
Tolera	<50% cómputo	<1/3 stake	f<n/3 (n≥3f+1)
Ej.	Bitcoin	Ethereum (2022)	CometBFT

Ethereum = SMR ($estado_{n+1}=\delta(estado_n,txs)$) + smart contracts en la EVM + **gas** (acota cómputo, corta loops; bugs inmutables: The DAO). **Conviene sólo si muchos escriben sin confiar y no hay tercero aceptable**; si no, BD replicada.

5 · ARQUITECTURAS (G8)

Receta diseño (E1,2,4): 1) supuestos 2) diagrama cajas+flechas 3) **por componente**: interfaz (operaciones) · procesamiento ("recibe X → hace Y → emite Z") · datos · nivel (UI/proc/datos) 4) **por conector**: protocolo (REST, RPC, pub/sub, BD compartida) + acoplamiento 5) justificar.
Estilos: en capas · objetos/servicios (REST: recursos + verbos, stateless) · pub/sub (event-based) · shared data space.

ref \ temp	Temporal acoplado	Temporal desacoplado
Ref. acoplado	directa (RPC, HTTP)	mailbox / cola
Ref. desacoplado	event-based (pub/sub)	shared data space (BD, tuplas)

Referencial = ¿el emisor conoce al receptor? Temporal = ¿vivos a la vez? **Pub/sub** por topic o por **contenido** (el publicador manda **valores**, el **suscriptor** pone el rango; filtra el middleware). Notificar con dato vs aviso + ir a buscar. Lease en suscripciones.
Condición A y B ⇒ el middleware no guarda estado ⇒ **controlador** que se suscribe, decide y publica comandos (regla en un lugar, actuadores tontos; punto único ⇒ replicar). Suscribirse a todo el rango para enterarse del "apagar" (E4).
Wrappers vs broker (E5): wrapper=adaptador de interfaz; reemplazar C,D por E ⇒ 2. N apps todos contra todos: **N(N−1)**; con broker **2N** (mejor desde N≥4, agregar = O(1); costo: punto central).
Idempotencia (E6): se pierde el OK sobre UDP ⇒ reintento ⇒ **transferencia doble** ⇒ **ID de transacción** (server detecta duplicado y re-responde). TCP no lo resuelve (caída entre procesar y responder: es de aplicación).
2-tier (a)→(e): corte UI / app / datos, de **thin** (sólo pantalla) a **fat** client. **Validación (E7)**: cliente (UX, menos viajes) **y** servidor (seguridad, el cliente se saltea).
Distribución **vertical** (capas en máquinas) vs **horizontal** (réplicas/particiones).

P2P

Overlay=red lógica sobre la física (1 enlace lógico = varios saltos).
Estructurado (DHT, Chord: anillo + finger table, **O(log N)**, hay que mantenerlo) vs **no estructurado**: **flooding** (TTL, $R(k)=d(d-1)^{k-1}$ msgs en salto k; rápido, robusto, carísimo) / **random walk** (S≈N/r pasos con r réplicas; barato, lento, no garantiza cubrir) / **super peers** (índice por grupo; débil si cae).

E8 sensores 5 pisos (BT al piso adyacente; nodo piso 1 WiFi+BT=**gateway**): overlay jerárquica, super peer por piso encadenados → AP; recolección por árbol **en paralelo + timeout + agregación**. Anillo: N msgs exactos pero O(N) latencia.
E9 1TB, 10000 piezas, 100 servers, 10 vecinos: $\mathbf{T_{pieza}=(2h+2)\cdot RTT/2+T_{tx}}$ (h=saltos búsqueda). Clase (100ms, 1Gbps): flooding≈3,5h · RW≈8,3h · super peer≈3,3h. Guía: RTT 200ms.

Cloud / Fog / Edge (E10)

Latencias (teórica): dispositivo <1ms · edge de acceso (AP/antena) 1–5 · fog/edge regional 5–20 · nube 50–150ms. **1ms RTT ≈ 100km** de fibra. **4 motivos para el borde**: **latencia** (tiempo real) · **BW** (filtrar/agregar antes de subir) · **autonomía** (sin Internet) · **regulación/privacidad**. Cloud = vista global, cómputo masivo. Tabla: componente | capa | procesa | datos | interfaces | por qué. Autos entre sí: **V2V broadcast**. Orquestación: dónde corre cada servicio + migrar (handoff). Problemas: placement (NP-difícil), migración, consistencia entre niveles, autonomía (CRDTs). Fog=jerarquía, edge=borde de acceso, cloudlet=DC en una caja.

6 · COMPUTO DISTRIBUIDO

Rasgos: concurrencia, sin reloj global, fallas independientes (sólo mensajes).
Modelos: tiempo (sínc / parcialmente sínc / asínc) · fallas (crash / crash-recovery / bizantina) · canales (confiables / pérdidas equitativas / arbitrarios).
Por qué distribuir: escalar, tolerar fallas, localidad, costo. Vertical tiene techo; horizontal exige coordinar.
Amdahl S(n)=1/((1−p)+p/n) → 1/(1−p). p=0,95: S(8)=5,9, S(64)=15,4, S(1024)=19,6 ⇒ tope **20×** (p=0,9 ⇒ 10×). Gustafson: si el problema crece con n, mejora. 10 000 servers MTBF 30 años ⇒ ~1 falla/día. **Cola**: $1-0{,}99^{100}\approx63\%$ (un pedido que toca 100 servers ve la latencia p99). Péndulo C↔D según costo computar/comunicar/latencia.
8 falacias: red confiable, latencia 0, BW ∞, red segura, topología fija, un admin, transporte gratis, red homogénea.
Dos generales: sin acuerdo garantizado sobre canal no confiable. **FLP**: asincrónico + 1 falla ⇒ no hay consenso determinístico garantizado. **CAP**: ante partición, C o A. **PACELC**: si P → A/C; si no (Else) → Latencia/Consistencia.
SMR: mismo orden de operaciones ⇒ mismo estado. Problema = acordar el orden del log = consenso (líder ordena). Tolerar f: crash **n≥2f+1** (Paxos, Raft), bizantinas **n≥3f+1** (PBFT): dos mayorías se intersectan ⇒ siempre hay testigo. Quórum **R+W>N** ⇒ toda lectura ve la última escritura (N=3,W=2,R=2 tolera 1 caída); bajar R/W compra latencia por consistencia.

Modelo	Garantiza	Ej.
Linealizable	cada op. instantánea entre inicio y fin (consenso por escritura)	Spanner, etcd
Secuencial	orden total común compatible con cada proceso	multiproc. ideal
Causal	si a pudo influir en b, todos ven a antes	COPS, chats
Eventual	si cesan escrituras, convergen	DNS, Dynamo, CRDT

Hacia abajo: menos coordinación/latencia, más disponibilidad, más conflictos para el programador. Cómo se vive con las imposibilidades: aleatoriedad, timeouts (sincronía parcial), detectores de falla. Nube: configuración (etcd, Chubby) = consenso + líder; MapReduce lleva cómputo a los datos.
a→b ("ocurrió antes"): mismo proceso con a primero; o a=envío y b=recepción del mismo msg; o transitividad. Si ninguno: **concurrentes**.

Lamport: (1) evento local C++; (2) enviar: C++ y adjuntar C; (3) recibir: **C=max(C,C_{msg})+1**. **a→b ⇒ C(a)<C(b), no al revés**. Orden total: desempatar por id de proceso.
Vectoriales: V[i]++ local; recibir: máx componente a componente + V[i]++. **a→b ⇔ V(a)<V(b)** (todas ≤ y una <); ninguno ≤ ⇒ **concurrentes**. Costo: n enteros por msg.

```
p1: a(1) —m1→p2  b(2) —m2→p3
p2: c(1) d(2) e=max(2,1)+1=3
p3:           f=max(0,2)+1=3
   a   b   c   d   e   f
L  1   2   1   2   3   3
V 100 200 010 020 130 201
```

C(c)=1<C(b)=2 pero son concurrentes (Lamport no lo ve; V: 010 vs 200 no comparables).

Relojes: cristal 20 ppm ≈ 1,7 s/día; NTP ms, PTP µs. **TrueTime** (Spanner): TT.now()=[earliest, latest], ancho 2ε. **ε=1ms+0,2ms/s·Δt** (Δt desde la última sync con GPS + atómicos, cada 30s; ~1–7ms; Δt=15s ⇒ ε=4ms, intervalo 8ms). API: TT.after(t)=t ya pasó seguro (t<earliest); TT.before(t)=no llegó (t>latest). **Commit wait**: s=TT.now().latest; no liberar locks ni responder hasta que TT.after(s) (≈2ε) ⇒ **consistencia externa** (T1 commit en A y después arranca T2 en B ⇒ s1<s2, para cualquier error ±ε). Convierte incertidumbre en latencia (2ε por commit, se solapa con Paxos). **vs Lamport**: Lamport ordena según causalidad que viaja en msgs; TrueTime ordena también eventos *sin* mensajes entre sí (anclado a hora real).

Teórica: definir en 1 línea → qué problema resuelve → trade-off/limitación → ejemplo. "X resuelve Y a cambio de Z".